Distributed Message Queue 2

Ad Click Event Aggregation 3

Hotel Reservation System 4

Distributed File Storage 5

Distributed Transactions 6

Distributed Key-Value Store 7

Payment System 8

Digital Wallet 9

Stock Exchange 10

Ride-sharing Services 11

Distributed Search 12

Distributed Cache 13

Distributed Job Scheduler 14

Distributed Logging 15

Distributed Counter 16

Distributed Pub-Sub 17

Distributed Rate Limiter 18

Distributed Notification Service 19

Distributed Ledger 20

Distributed Lock 21

Distributed Unique ID Generator 22

Distributed Leaderboard 23

Distributed Web Crawler 24

Distributed Ad Serving 25

Distributed Text Search 26

Distributed Log Monitoring 27

28. Digital Wallet

Payment platforms usually provide a digital wallet service to clients, so they can store money in the wallet and spend it later. For example, you can add money to your digital wallet from your bank account and use it to pay for products online, or you can ask the system to pay utility the money in your wallet. Figure 1 shows a simple digital wallet.

Figure 1: Cross-border balance transfer

Balances can be used to charge the customer in digital wallet applications that supports the cross-border balance transfer operation. In this chapter, we will design a digital wallet that supports cross-border balance transfer.

Step 1 - Understand the Problem and Establish Design Scope

Interviewer: Let's focus on the balance transfer operation. Do you need to support cross-currency transfer?

Candidate: Yes, we need to support cross-currency transfer.

Candidate: How many transactions per second (TPS) does the system need to support?

Interviewer: Let's assume 1,000,000 TPS.

Candidate: A digital wallet has strict requirements for correctness. Can we assume transaction guarantees (C) are sufficient?

Interviewer: That sounds good.

Candidate: Do we need to prove correctness?

Interviewer: Yes, that is a great question. Correctness is usually only verifiable when a transaction is completed. One way to verify is to compare our internal records with statements from banks. The limitation is that it takes days for statements to arrive, and discrepancies can not be fully reconciled even after that. Therefore, we would like to design a system with reproducibility meaning we could always reconstruct the system state by replaying the data from the beginning.

Candidate: Can you explain the availability requirement in detail?

Interviewer: Sounds good.

Candidate: Is 99.99% a good enough availability requirement?

Interviewer: Yes, it's out of scope.

In summary, our digital wallet needs to support the following:

- Support balance transfer operation between two digital wallets.
- Support cross-currency.
- Reliability, or exact ACIDity.
- Support transactions.
- Support reproducibility.

Back-of-the-envelope estimation

When we talk about TPS, the ledger is a fundamental database left as great. Today, a relational database running on a typical latest server can easily support 10,000 TPS. Therefore, we need to shard the ledger database to meet the requirement of 1,000,000 TPS. Sharding is one of the popular transactional database scaling solutions, sharding in distributed systems is not trivial, we will dive into it later. Let's assume 100 nodes need 1,000 database shards.

Cross-currency transfer is considerably slightly inaccurate. Each transfer command requires two operations, debiting money from one account and depositing money to the recipient's account. Assuming the system supports 1,000,000 TPS, the system actually needs to handle up to 2 million TPS, which means we need 2,000 nodes.

Table 1 shows the total number of nodes required when the "per-node TPS" (the TPS a single node can handle) changes, fluctuating hardware towards the years. We more transactions a single node can handle per second, the lower the total number of nodes required, indicating lower hardware cost. We will use the "per-node TPS" in the following calculation of a single node cost.

Per-node TPS	Node Number
10,000	200
5,000	2,000
10,000	200

Table 1: Changing Ethereum gas limit with hardware upgrade

Step 2 - Propose High-Level Design and Get Buy-In

In this section, we will discuss the following:

- API design
- Three high level designs:
  - Simple in-memory solution
  - Database based distributed transaction solution
  - Event-sourcing based with reproducibility

API Design

We will use the RESTful API convention. For this interview, we only need to support one API.

	Detail
URL	/v1/wallet/balance_transfer
Request parameters are:	

Field	Description	Type
from_account	The debit account	string
to_account	The credit account	string
amount	The amount of money	string
currency	The currency type	string (ISO 4217 [2])
transaction_id	A used-by idempotent	uuid

Sample response body:

```
{
"balance": "balance",
"transaction_id": "transaction_id"
}
```

One thing worth mentioning is that the data type of the "amount" field is "string" rather than "double". We explained the reasoning in the Payment System chapter.

In practice, many people will choose float or double representation of numbers because it is supported by almost every programming language and platform. Floating-point numbers are not recommended for financial applications.

Distributed transactions

The wallet application maintains an account balance for every user, a ledger database structure to represent the cause/balance relationship is a map, which is also called a hash table (map) or key-value store.

For example, if user a has initial balance of $10, user a's balance is $10. We change to handle 1 dollar PIN. We need to set up a cluster of ledger servers to handle the load. For simplicity, we only show one database in the following diagram.

To distribute the cross-sales among different hyperlinks, we could calculate the hash value of the key and divide it by 4. The remainder is the destination of the partition. The production level more complicated than this process.

```
string accountId = "a";
int hash = accountId.hashCode();
int partitionNumber = accountId%4 // partitionNumber;
```

The number of partitions and addresses of these hosts can be stored in a centralized place. We could use Zookeeper [3] as a highly available configuration storage service.

The final component of the solution is a service that handles the transfer commands. We call it the wallet service and it has several key responsibilities:

1. Receives the transfer command.
2. Validates the transfer command.
3. If the command is valid, it updates the balance of the two balance fields (one for the source account and another for the destination account).

The wallet service is stateless. It is easy to scale horizontally. Figure 2 shows the in-memory solution.

Figure 2: In-memory solution

In this example, we have 3 fields inside. There are three clients, A, B, and C. Their account balances are evenly spread across these three servers. The wallet service receives two transfer commands: A pays B $1 and A pays C $2. Unfortunately, the wallet service is a stateless service, it can't hold the balance information. The data, stored by A, B and C, is lost when the server is down. It is also hard for us to scale. The Redis cluster that contains shard 0 is down, the transfer command that involves shard 0 is unavailable. For client B, the wallet service adds $1 to the account.

Candidate: In the design, account balances are spread across multiple Redis nodes. Distributed is used to maintain the sharding information. Is the wallet service responsible for persisting information in Redis nodes? Does the Redis cluster ensure that it can handle the balance accordingly.

Interviewer: This design only has 3 clients and is not our concerning requirement. The wallet service updates two Redis nodes for each transfer. The wallet service receives two transfer commands: A pays B $1 and A pays C $2. Unfortunately, the wallet service crashes after it has gone through and updated the account's balance. B thinks it should receive $1 and transfer information. The two updates need to be in a single atomic transaction.

Distributed transactions

Database sharding

How do we make the updates to two different storage nodes atomic? The first step is to replace each Redis node with a transactional database, such as a relational database or a key-value store that supports ACIDity. In this case, Redis is, and if we partitioned the 3 relational databases, rather than 3 Redis nodes.

Figure 3: Relational database

Using transactional databases only solves part of the problem. As mentioned in the last section, it is very likely that one transfer command involves two operations, debiting money from one account and crediting money to another account. Both operations must succeed or all exactly the same or fail. The wallet service received right after it updated the first account balance, how can we make sure the second account will be updated as well?

Distributed transaction: two-phase commit

In a distributed system, a transaction may involve multiple processes or multiple nodes. To make a transaction atomic, the distributed transaction service [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [16] [17] [18] [19] [20] [21] [22] [23] [24] [25] [26] [27] [28] [29] [30] [31] [32] [33] [34] [35] [36] [37] [38] [39] [40] [41] [42] [43]

Definition

There are four important terms in event sourcing.

1. Command
2. Event
3. State
4. State machine

Command

A command is the intended action from the outside world. For example, if we want to transfer $10 from client A to client C, this means, transfer $10 from client A.

A event sourcing, it is very important that everything has an order. So command will be processed only one by a PRC(that is, first-last queue).

Event

Command is an intention and not a fact because some commands may be invalid and cannot be fulfilled. For example, the transfer operation will fail if the account balance is less than the transfer amount.

A command must be validated before as an ordering event. Once the command passes the validation, it is valid and must be fulfilled. The result of the fulfillment is called an event.

Events are immutable and are persisted in an append-only event log.

1. Events must be executed because they represent a validated fact. In practice, we usually use the past tense for an event. If the command is Transfer $10 from A to C, the corresponding event would be Transferred $10 from A to C.
2. Commands may contain randomness or CE, but events must be deterministic. Events represent historical facts.

There are two important properties of the event generation process.

1. One command produces one event. Command is a request, events are or more events.
2. Event generation may contain randomness, relationship. It is not guaranteed that a command always generates the same correctly. The randomness may come from the business logic, third-party services, or even the current time. The state machine must be deterministic.

The order of events must follow the order of commands. So events are stored in a FIFO queue, and

State

State is what drive the system to make the next event is applied. In the state machine, the state is a function of all state-accounts, which can be represented with a map data structure. The key is the account name or ID, and the value is the account balance. Key-value stores are usually used to store the state structure. The historical database can also be stored as a key-value store, where keys are primary keys and values are state.

State machine

A state machine drives the event sourcing process. It has two major functions.

1. Validate incoming commands.
2. Apply event to update state.

Event sourcing requires the behavior of the state machine to be deterministic. Therefore, the state machine itself should never contain any randomness. For example, it should never read anything from the system clock, or use any random numbers. When it applies an event to state, it should always generate the same result.

Event sourcing decouples the state machine from the outside world. The state machine is responsible for connecting the command to an event and for applying the event to update state machine has no direct interfaces, but, are usually driven by message-based interfaces, one for validating commands and the other for applying events.

Figure 1: State view of event sourcing

Figure 2: Event view of event sourcing

Wallet service example

For the wallet service, the state is the account balance transfer requests. These commands are put into a FIFO queue. One popular choice for the command queue is Kafka. The command queue is stored in Kafka 0.

Figure 3: Command queue

Let us assume the state also processed events is stored in a relational database. The state machine examines each command one by one in FIFO order. For each command, it checks whether the account has a sufficient balance. If yes, the state machine generates an event representing the transfer request and updates the state. For example, if the state machine generates two events, "A" and "C,A#", the

Figure 4 shows how the state machine works in 5 steps.

1. Read commands from the command queue.
2. Read balance state from the database.
3. Validate the command. If it is valid, generate two events for each of the accounts.
4. Update balance state.
5. Apply the Event by updating the balance state in the database.

Figure 4: State machine

Reproducibility

The most important advantage that event sourcing can offer is reproducibility.

Because the state machine is deterministic, we can always rebuild the state from the event log. Rebuilding account balance (the state) into the database is a difficult or even, since the current state can change dramatically. Reproducibility helps developers to troubleshoot or test during the outage operations. In the event sourcing design, all changes are saved that are immutable history. This database is only used as an updated view of what balance looks like at any given point in time.

We could always reconstruct historical balance states by replaying the events from the very beginning. Because the event log is immutable and the state machine logic is deterministic, it is guaranteed that the historical states generated from each replay are the same.

Figure 5 shows how to reproduce the states of the wallet service by replaying the events.

Figure 5: Reproducibility

Reproducibility helps us answer the difficult questions that the auditors ask at the beginning of the section. We repeat the questions here.

1. Do we know the account balance at any point in time?
2. How do we know the current account balance is correct?
3. How do we prove the system logic is correct after a code change?

For the first question, we could answer it by replaying events from the start, up to the point in time where we would like to know the account balance.

For the second question, we could verify the correctness of the account balance by recalculating it from the event list.

For the third question, we can run different versions of the code against the same event and verify that the results are identical.

Because of the audit capability, event sourcing is often chosen as the de facto solution for the wallet service.

Command query responsibility segregation (CQRS)

So far, we have designed the wallet service to move money from one account to another efficiently. However, the client still does not know what the account balance is. There needs to be a way to provide information to the client, which is the read path. In event sourcing framework, we know what the state is.

Internally, we can calculate the current state of the database (historical state) and share it with the outside world. Event sourcing ensures consistency between the state and the event log.

Rather than publishing the state (aka mutations), event sourcing publishes all the events. The external world could rebuild any customized view itself. This design pattern is called CQRS.

In CQRS, there is one state machine responsible for the write part of the state, but there can be many read only state machines, which are responsible for building up the read state. The state machine is the source of truth and is the only place that can make changes.

These read only state machines can derive different state representations for their read operations. For example, clients may want to know the account balance in USD, EUR, or CNY. The read only state machine can provide the balance in the requested currency. The read only state machine could write to a specific time period to help investigate issues or to provide audit trails or analytics. The state machine, on south-bound side, could help to accounts the financial records.

The read only state machine rebuilding is a batch process, which will always catch up. The architecture design is eventually consistent.

Figure 16 shows a classic CQRS architecture.

Figure 16: CQRS architecture

Candidate: In this design, we can event sourcing architecture to make the write portion scalable. All valid business records are stored as immutable facts and are which could be used for analytics or troubleshooting.

Interviewer: That's great. But this event sourcing architecture put upon very families one event at a time and it needs to communicate with external systems. Can we make it faster?

Step 3 - Design Deep Dive

In this section, we dive into the technique for achieving high performance, reliability, and scalability.

High-performance event sourcing

In the earlier examples, we used Kafka as the command and event store, and the database as a state store. Let's explore some optimizations.

File-based command and event list

The first optimization is to store command and events in a local disk, rather than in a remote store like Kafka. This avoids round trips across the network. The event list can be persisted in a local disk, which allows sequencing, appending to a sequential write operation, which is generally very fast. Kafka uses this approach to persist its event log. Because the command and event list are local to the state machine, it can be accessed by the OS file system cache. OS, sequential write accesses are faster than random accesses in some cases.

The second optimization is to cache recent commands and events in memory. As we explained before, we process commands and events right after they are persisted. We may cache them in memory to avoid the cost of reading back from the disk.

We are going to explore some implementation details. A technique called Virtual Z[1] is great for implementing the optimizations mentioned above. Virtual Z is a distributed event sourcing framework. It manages the command and event lists and the state machine. The application logic is implemented as a gRPC service. Virtual Z manages the command and event lists and the state machine. For approach and the operations. For approach only the operations, it is almost guaranteed that all data are stored in memory, which is very fast.

Figure 20 shows the file-based command and event storage.

Command List
Event List
Virtual Z
Virtual Z
Virtual Z

4. Partition 1's Raft leader receives the A-$1 command and stores it in the command list. It then validates the command. If it is valid, it is converted into an event. The Raft consensus algorithm is used to synchronize data across different nodes. The event (deducting $1 from A's account balance) is executed after synchronization is complete.
5. After the event is synchronized, the event sourcing framework of Partition 1 synchronizes the data to the read path using CQRS. The read path reconstructs the state and the status of execution.
6. The read path of Partition 1 pushes the status back to the caller of the event sourcing framework, which is the Saga coordinator.
7. Saga coordinator receives the success status from Partition 1.
8. The Saga coordinator creates a record, indicating the operation in Partition 1 is successful, in the phase status table.
9. Because the first operation succeeds, the Saga coordinator executes the second operation, which is C+$1. The coordinator sends C+$1 as a command to Partition 2 which contains account C's information.
10. Partition 2's Raft leader receives the C+$1 command and saves it to the command list. If it is valid, it is converted into an event. The Raft consensus algorithm is used to synchronize data across different nodes. The event (add $1 to C's account) is executed after synchronization is complete.
11. After the event is synchronized, the event sourcing framework of Partition 2 synchronizes the data to the read path using CQRS. The read path reconstructs the state and the status of execution.
12. The read path of Partition 2 pushes the status back to the caller of the event sourcing framework, which is the Saga coordinator.
13. The Saga coordinator receives the success status from Partition 2.
14. The Saga coordinator creates a record, indicating the operation in Partition 2 is successful in the phase status table.
15. At this time, all operations succeed and the distributed transaction is completed. The Saga coordinator responds to its caller with the result.

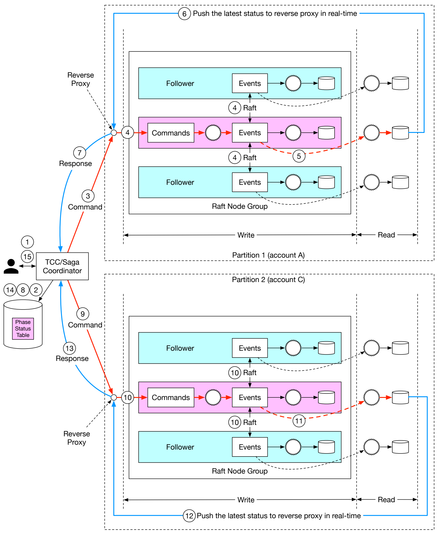


Figure 29 Final design in a numbered sequence

Step 4 - Wrap Up

In this chapter, we designed a wallet service that is capable of processing over 1 million payment commands per second. After a back-of-the-envelope estimation, we concluded that a few thousand nodes are required to support such a load.

In the first design, a solution using in-memory key-value stores like Redis is proposed. The problem with this design is that data isn't durable.

In the second design, the in-memory cache is replaced by transactional databases. To support multiple nodes, different transactional protocols such as 2PC, TC/C, and Saga are proposed. The main issue with transaction-based solutions is that we cannot conduct a data audit easily.

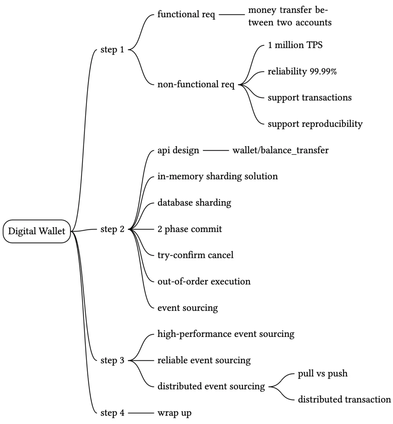
Next, event sourcing is introduced. We first implemented event sourcing using an external database and queue, but it's not performant. We improved performance by storing command, event, and state in a local node.

A single node means a single point of failure. To increase the system reliability, we use the Raft consensus algorithm to replicate the event list onto multiple nodes.

The last enhancement we made was to adopt the CQRS feature of event sourcing. We added a reverse proxy to change the asynchronous event sourcing framework to a synchronous one for external users. The TC/C or Saga protocol is used to coordinate Command executions across multiple node groups.

Congratulations on getting this far! Now give yourself a pat on the back. Good job!

Chapter Summary



Reference materials

[1] Transactional guarantees: https://docs.oracle.com/cd/E17275_01/html/programmer_reference/rep_trans.html

[2] TPC-E Top Price/Performance Results: http://tpc.org/tpce/results/tpce_price_perf_results5.asp?resulttype=all

[3] ISO 4217 CURRENCY CODES: https://en.wikipedia.org/wiki/ISO_4217

[4] Apache Zookeeper: https://zookeeper.apache.org/

[5] Martin Kleppmann (2017). Designing Data-Intensive Applications. O'Reilly Media.

[6] X/Open XA: https://en.wikipedia.org/wiki/X/Open_XA

[7] Compensating transaction: https://en.wikipedia.org/wiki/Compensating_transaction

[8] SAGAS, HectorGarcia-Molina: https://www.cs.cornell.edu/andru/cs711/2002fa/reading/sagas.pdf

[9] Evans, E. (2003). Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley Professional.

[10] Apache Kafka. https://kafka.apache.org/

[11] CQRS: https://martinfowler.com/bliki/CQRS.html

[12] Comparing Random and Sequential Access in Disk and Memory: https://deliveryimages.acm.org/10.1145/1570000/1563874/jacobs3.jpg

[13] mmap: https://man7.org/linux/man-pages/man2/mmap.2.html

[14] SQLite: https://www.sqlite.org/index.html

[15] RocksDB. https://rocksdb.org/

[16] Apache Hadoop: https://hadoop.apache.org/

[17] Raft: https://raft.github.io/

[18] Reverse proxy: https://en.wikipedia.org/wiki/Reverse_proxy

Mark as Complete

Partner With Us
Be an Affiliate
Become a Contributor

Company & Legal
Our Team
Newsletter
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information

Support
hi@bytebytego.com
Report a Bug

Resources
YouTube
Visual Dev Guides
Find Jobs
Prepare for Coding Interviews

Ask Alex

Copyright ©2022-2026 ByteByteGo Inc. All rights reserved.